DATA 427 Group Project: Spark ML on the KDD Dataset

Jason Pollock, Fabian Day, Bex Campbell-Redl

1 Implement two programs that apply Spark's Decision Tree algorithm and Logistic Regression algorithm to the provided KDD dataset. Run these programs 10 times on the school's Hadoop cluster, each using a different seed to split the dataset into a training and a test set.

1.1 Describe the two programs using pseudo-code

1.1.1 Logistic Regression:

Algorithm 1 Spark Logistic Regression

Parse the input/output directories and number of runs
Initialise the Spark session and load the data from the provided input directory

Setup the pipeline as follows:
Index required columns
One-hot encode required columns
Assemble into feature columns
Append parameterised logistic regression model as the final stage.
for Number of runs **do**
    Set seed
    The data into train/test
    Fit the LR model on the train data
    Evaluate the models performance on the training and testing split, and record results
end for
Display results

1.1.2 Decision Tree

Algorithm 2 Spark Decision Tree

Parse the input/output directories and number of runs
Initialise the Spark session and load the data from the provided input directory
Generate schema
Setup the pipeline as follows:
Index required columns
Assemble into feature columns
Append parameterised DecisionTree model as the final stage.
Run the pipeline:
Split the test/training data into num_runs instances
Foreach training data, train a model
Evaluate the results:
Output the DecisionTree structure for the first tree
Foreach (model, training data) pair, generate predictions
Foreach (model, testing data) pair, generate predictions
Foreach training prediction, generate an accuracy row
Foreach test prediction, generate an accuracy row
Produce a CSV with the aggregate data by test/train
Produce a CSV with the per-run data
Display results

1.2 Describe how to install and run the two programs step by step in a Readme.txt file.

*** README**

group: 22

authors: Bex Campbell-Redl, Fabian Day, Jason Pollock

email: `campberebe3@myvuw.ac.nz`, `dayfabi@myvuw.ac.nz`, `pollocjaso@myvuw.ac.nz`

How to Install and Run Group22's Decision Tree and Logistic Regression code on the Hadoop cluster

Important note on Makefile: This project uses a makefile. A makefile is the most efficient way to ensure all required setup, admin and installation tasks have been successfully executed before attempting to run code. This is particularly important when working in a group to ensure everyone has the same setup when running or writing code. The makefile for this includes hadoop setup, java8, kinit and sparkclasspath setup. Please read the makefile for further information.

Part 1: Setup & Environment

1. Ensure that you have an ECS user account and can either access the lab or SSH in via username@entry.ecs.vuw.ac.nz and then into a cluster machine
2. Ensure that you have a user directory on the HDFS cluster (if you do not have one, create one)
3. Download the submitted Tarball into a folder on your ECS desktop
4. Extract the tarball (alternatively you can clone our git repo)
5. cd into the extracted folder

Part 2: Understanding the Directory Layout

Directories:

- `SparkWordCount/input` - the dataset extracted and ready for use
- `SparkWordCount/bin` - where the code is placed
- `SparkWordCount/staged_output` - where local runs place their output instead of HDFS
- `DecisionTree/input` - the dataset extracted and ready for use
- `DecisionTree/bin` - where the code is placed
- `DecisionTree/staged_output` - where local runs place their output instead of HDFS
- `LogisticRegression/input` - the dataset extracted and ready for use
- `LogisticRegression/bin` - where the code is placed
- `LogisticRegression/staged_output` - where local runs place their output instead of HDFS
- `ecs_hadoop_env` - the ECS cluster Hadoop makefile variables
- `ecs_spark_env` - the ECS cluster Spark makefile variables
- `local_hadoop_env` - the Hadoop makefile variables for local execution
- `local_spark_env` - the Spark makefile variables for local execution

Part 3: How to run the code

The makefile supports two execution modes, ecs and local, picked via a ENV environment variable. ”ecs” is the default.

The makefile will ensure proper configured access to kerberos, HDFS and Spark before attempting to launch the job.

This is configured to run via makefile.

Part 3a: Decision Tree

- Run the Decision Tree program:
  - `$ make submit_DecisionTree`
- The output will be placed in:
  - `DecisionTree/output`
- View the output:
  - `$ cat DecisionTree/output/*.csv`
- To clean the files when complete:
  - `$ make clean_DecisionTree`

Part 3b: Logistic Regression

- Run the Logistic Regression program:
  - `$ make submit_LogisticRegression`
- The output will be placed in:
  - `LogisticRegression/output`
- View the output:
  - `cat LogisticRegression/output/*.csv`
- To clean the files when complete:
  - `$ make clean_LogisticRegression`

Optional Variant modes:
Runs spark wordcount

- `$ make submit_SparkWordCount`

To Run all modes

- `$ make all`

The output(s) will be placed in:

- `DecisionTree/output`
- `LogisticRegression/output`
- `SparkWordCount/output`

- `$ cat DecisionTree/output/*.csv`
- `$ cat LogisticRegression/output/*.csv`
- `$ cat SparkWordCount/output/*.csv`

To clean the files when complete:

- `$ make clean_DecisionTree`
- `$ make clean_LogisticRegression`
- `$ make clean_SparkWordCount`

or

- `$ make clean`

1.3 Report the training and test results including the max, min, average accuracy and the standard deviation obtained from the 10 runs, and the running time of each program.

1.3.1 Decision Tree:

Table 1: Summary of decision tree accuracy over 10 runs.

Metric	Average	Std. dev.	Min	Max
Training accuracy (%)	94.62	0.15	94.39	94.94
Test accuracy (%)	94.57	0.21	94.37	95.06

1.3.2 Logistic Regression:

Table 2: Summary of logistic regression accuracy over 10 runs.

Metric	Average	Std. dev.	Min	Max
Training accuracy (%)	94.35	0.11	94.21	94.50
Test accuracy (%)	94.15	0.17	93.93	94.37

Table 3: Summary of model test accuracy and total running time over 10 runs.

Program	Average acc. (%)	Std. dev.	Min acc. (%)	Max acc. (%)	Total time (s)
Decision Tree	94.57	0.21	94.37	95.06	65.37
Spark Logistic Regression	94.15	0.17	93.93	94.37	129.29

1.4 Appendix:

Table 4: Decision tree results for each seeded run.

Run	Seed	Train count	Test count	Train acc. (%)	Test acc. (%)
1	221	33372	14364	94.49	94.40
2	222	33414	14322	94.71	94.40
3	223	33434	14302	94.67	94.39
4	224	33373	14363	94.61	94.37
5	225	33484	14252	94.53	94.56
6	226	33351	14385	94.59	94.68
7	227	33502	14234	94.66	94.65
8	228	33167	14569	94.39	94.63
9	229	33494	14242	94.94	95.06
10	230	33482	14254	94.59	94.61

Table 5: Logistic regression results for each seeded run.

Run	Seed	Train count	Test count	Train acc. (%)	Test acc. (%)	Time (s)
1	1	33402	14334	94.35	94.06	10.09
2	2	33429	14307	94.24	94.07	7.93
3	3	33571	14165	94.36	94.35	7.09
4	4	33434	14302	94.50	94.37	7.04
5	5	33314	14422	94.47	94.03	6.63
6	6	33570	14166	94.30	94.16	6.61
7	7	33431	14305	94.30	93.93	6.69
8	8	33535	14201	94.21	94.37	6.47
9	9	33489	14247	94.27	94.20	6.40
10	10	33357	14379	94.50	93.95	6.37

1.4.1 Example of trained decision tree structure from a run

DecisionTreeClassificationModel: depth=5, numNodes=51, numClasses=2, numFeatures=41
if feature 4 $\leq$ 30.5 **then**
  if feature 35 $\leq$ 0.005 **then**
    if feature 5 $\leq$ 130.5 **then**
      if feature 1 $\leq$ 0.5 **then**
        Predict: 0.0
      else
        if feature 34 $\leq$ 0.015 **then**
          Predict: 1.0
        else
          Predict: 0.0
        end if
      end if
    else
      if feature 2 $\leq$ 15.5 **then**
        Predict: 1.0
      else
        if feature 32 $\leq$ 132.5 **then**
          Predict: 0.0
        else
          Predict: 1.0
        end if
      end if
    end if
  else
    if feature 32 $\leq$ 168.5 **then**
      Predict: 0.0
    else
      if feature 4 $\leq$ 0.5 **then**
        if feature 2 $\leq$ 52.5 **then**
          Predict: 1.0
        else
          Predict: 0.0
        end if
      else

```
                    if feature 36 ≤ 0.015 then
                        Predict: 1.0
                    else
                        Predict: 0.0
                    end if
                end if
            end if
        end if
    else
        if feature 39 ≤ 0.005 then
            if feature 1 ≤ 1.5 then
                if feature 0 ≤ 29.5 then
                    Predict: 1.0
                else
                    if feature 2 ≤ 17.5 then
                        Predict: 0.0
                    else
                        Predict: 1.0
                    end if
                end if
            else
                if feature 4 ≤ 195.5 then
                    if feature 30 ≤ 0.685 then
                        Predict: 1.0
                    else
                        Predict: 0.0
                    end if
                else
                    if feature 4 ≤ 1518.5 then
                        Predict: 0.0
                    else
                        Predict: 1.0
                    end if
                end if
            end if
        else
            if feature 36 ≤ 0.005 then
                if feature 4 ≤ 14714.5 then
                    if feature 5 ≤ 608.5 then
                        Predict: 0.0
                    else
                        Predict: 1.0
                    end if
                else
                    if feature 33 ≤ 0.615 then
                        Predict: 1.0
                    else
                        Predict: 0.0
                    end if
                end if
            else
```

```
            if feature 0 ≤ 2.5 then
                if feature 7 ≤ 0.5 then
                    Predict: 1.0
                else
                    Predict: 0.0
                end if
            else
                if feature 5 ≤ 130.5 then
                    Predict: 0.0
                else
                    Predict: 1.0
                end if
            end if
        end if
    end if
end if
```